Section 11: Linked Lists

This section is part of the 5-hour LeetCode DSA Foundations course by Mukul Raina

- YouTube Course: https://youtu.be/YJMBv4Q2Tg8?si=Q1vDzusUJf753GG3
- YouTube Channel: https://www.youtube.com/@mukulraina
- Instructor LinkedIn: https://www.linkedin.com/in/mukulraina/

SLIDE 99: Section Intro, Linked Lists

SLIDE 100: Linked List Fundamentals and the Dummy Node

SLIDE 101: Reversing a Linked List

SLIDE 102: Cycle Detection with Floyd's Algorithm

SLIDE 102B: Finding the Cycle Start

SLIDE 103: Fast-Slow Pointers and Gap Pointers

SLIDE 104: Merging Sorted Linked Lists

SLIDE 105: Worked Example: Reverse Nodes in K-Group

SLIDE 106: Wrong vs Right: Linked List Mistakes

SLIDE 107: How to Present Linked List Solutions in an Interview

SLIDE 108: Interview Practice: Linked Lists

SLIDE 109: Section Cheat Sheet

SLIDE 99: Section Intro, Linked Lists

Section 11: Linked Lists

When a problem hands you a linked list, the data structure itself is the challenge. There is no random access, no binary search, and you cannot know the length without a full traversal. Every operation requires careful pointer management, and a single miswired pointer loses the rest of the list with no way to recover it.

Linked list problems test whether you can manipulate pointers accurately under pressure. The logic is rarely complex. The difficulty is in getting every .next assignment correct, in the right order, without losing references.

What We'll Cover

1. Linked list fundamentals and operation costs vs arrays
2. The dummy node technique for eliminating head-related edge cases

3. Reversing a linked list (the single most tested operation)
4. Floyd's cycle detection algorithm
5. Fast-slow pointers for finding the middle and gap pointers for Nth from end
6. Merging sorted linked lists
7. Worked example: Reverse Nodes in K-Group (combining reversal with group logic)

Where This Section Connects

Section	Connection
Two Pointers (Section 6)	Fast-slow pointers are the linked list variant of same-direction two pointers
Sorting (Section 9)	Merge sort on linked lists reuses the merge operation from this section
Stacks (Section 10)	Some linked list problems (palindrome check) use a stack to compare first half with reversed second half
Recursion (Section 12)	Linked list reversal and merging have recursive variants. Iterative is preferred for O(1) space

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 100: Linked List Fundamentals and the Dummy Node

Node Structure

Every linked list node stores a value and a reference to the next node. The last node's next reference is null, marking the end of the list.

```
class ListNode {
    int val;
    ListNode next;
    ListNode(int val) { this.val = val; }
}
```

Operation Complexity: Linked List vs Array

Operation	Linked List	Array	Why the Difference
Access by index	O(n)	O(1)	Must traverse from head
Search	O(n)	O(n)	Both require linear scan

Operation	Linked List	Array	Why the Difference
Insert at head	O(1)	O(n)	No shifting needed
Insert at middle	O(1) after finding position	O(n)	No shifting, just rewire pointers
Delete at head	O(1)	O(n)	No shifting needed
Delete at middle	O(1) after finding position	O(n)	No shifting, just rewire pointers
Find length	O(n)	O(1)	Must traverse entire list

The trade-off: linked lists give O(1) insert/delete at any known position (no shifting), but lose O(1) random access. Arrays give instant access to any index but pay O(n) for insertions and deletions in the middle.

The Dummy Node Technique

The most common source of bugs in linked list code is special-casing the head node. When the head might be removed, replaced, or when inserting before the head, the logic diverges from the general case.

The dummy node eliminates this entirely. Create a placeholder node whose next points to the real head. Perform all operations starting from the dummy. Return dummy.next as the new head.

Without dummy (error-prone):

```
public ListNode removeElements(ListNode head, int val) {
    while (head != null && head.val == val) {
        head = head.next;
    }

    ListNode curr = head;
    while (curr != null && curr.next != null) {
        if (curr.next.val == val) {
            curr.next = curr.next.next;
        } else {
            curr = curr.next;
        }
    }

    return head;
}
```

With dummy (clean):

```java
public ListNode removeElements(ListNode head, int val) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;

    ListNode curr = dummy;
    while (curr.next != null) {
        if (curr.next.val == val) {
            curr.next = curr.next.next;
        } else {
            curr = curr.next;
        }
    }

    return dummy.next;
}
```

The dummy version has no special case for the head. The while loop handles every node uniformly because curr always starts one position before the first real node. Use a dummy node whenever the head might change during the operation.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 101: Reversing a Linked List

The Most Tested Linked List Operation

Reversal appears as a standalone problem and as a subroutine inside dozens of other problems (reverse in groups, reverse between positions, palindrome check, reorder list). If you can reverse a linked list cleanly and quickly, you can solve most linked list problems.

The Three-Pointer Technique

Maintain three references: prev (the already-reversed portion), curr (the node being processed), and next (saved reference to the rest of the list). At each step, four operations in exact order:

1. Save curr.next into next (before we overwrite it)
2. Rewire curr.next to point to prev (reverse the link)
3. Advance prev to curr

4. Advance curr to next

```java
public ListNode reverseList(ListNode head) {
    ListNode prev = null;
    ListNode curr = head;

    while (curr != null) {
        ListNode next = curr.next;
        curr.next = prev;
        prev = curr;
        curr = next;
    }

    return prev;
}
```

Why order matters: step 1 MUST happen before step 2. Once you execute curr.next = prev, the original curr.next is gone. If you did not save it first, the rest of the list is lost permanently.

Trace through 1 → 2 → 3 → 4 → null:

Step	prev	curr	next (saved)	Action	List State
Start	null	1	-	-	1 → 2 → 3 → 4
1	null	1	2	1.next = null	null ← 1 2 → 3 → 4
2	1	2	3	2.next = 1	null ← 1 ← 2 3 → 4
3	2	3	4	3.next = 2	null ← 1 ← 2 ← 3 4
4	3	4	null	4.next = 3	null ← 1 ← 2 ← 3 ← 4
End	4	null	-	curr is null, exit	4 → 3 → 2 → 1 → null

Return prev, which points to 4 (the new head). Time O(n), space O(1).

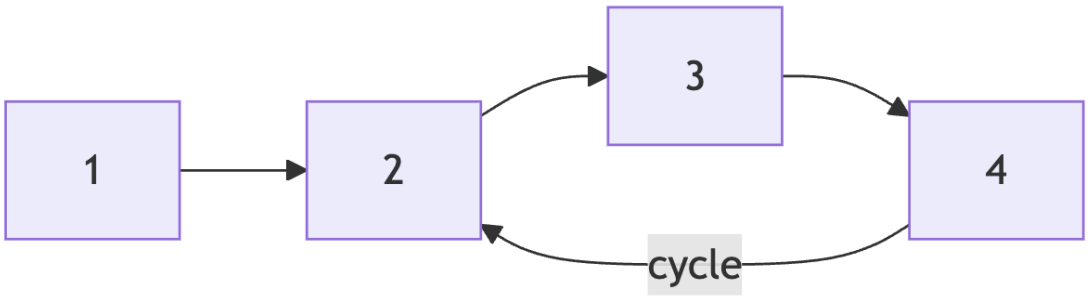
Interview Tip: Practice writing this code from memory until it takes under 60 seconds. In interviews, reversal is often a subroutine inside a harder problem, and spending 5 minutes on the subroutine leaves too little time for the main logic.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

SLIDE 102: Cycle Detection with Floyd's Algorithm

The Problem

A linked list has a cycle if some node's next pointer points back to an earlier node, creating an infinite loop. Given a linked list, determine whether a cycle exists. You cannot modify the list, and O(1) space is required (ruling out a HashSet of visited nodes).



In this list, node 4 points back to node 2, creating a cycle through nodes 2, 3, 4.

Floyd's Tortoise and Hare Algorithm

Use two pointers: slow moves one step at a time, fast moves two steps. If there is no cycle, fast reaches null. If there is a cycle, fast enters the cycle first and slow enters later. Once both are in the cycle, the gap between them decreases by exactly 1 at every step, so they must eventually meet.

```
public boolean hasCycle(ListNode head) {
    ListNode slow = head;
    ListNode fast = head;

    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;

        if (slow == fast) return true;
    }

    return false;
}
```

Time O(n), space O(1).

Why They Must Meet

Once both pointers are inside the cycle, think of the gap between them. Fast gains 1 position per step (moves 2, slow moves 1, net gain = 1). If the gap starts at d within a cycle of length c, after d steps the gap becomes 0. They meet. This is guaranteed regardless of where the cycle starts or how long the tail is.

Trace through 1 → 2 → 3 → 4 → 5 → 3 (cycle at node 3):

Step	slow	fast	Met?
Start	1	1	No
1	2	3	No
2	3	5	No
3	4	4	Yes

After 3 steps, slow and fast both point to node 4. Cycle detected. The algorithm does not tell us where the cycle starts, only that one exists. Finding the start requires a second phase, covered in the next slide.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 102B: Finding the Cycle Start

Phase 2: Locating Where the Cycle Begins

After slow and fast meet inside the cycle, reset one pointer to the head. Advance both pointers at speed 1 (one step at a time). The node where they meet again is the cycle start.

```
public ListNode detectCycle(ListNode head) {
    ListNode slow = head;
    ListNode fast = head;

    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;

        if (slow == fast) {
            ListNode entry = head;
            while (entry != slow) {
```

```
                entry = entry.next;
                slow = slow.next;
            }
            return entry;
        }
    }

    return null;
}
```

Why This Works

Let the distance from head to cycle start be F (the tail length). Let the distance from cycle start to the meeting point be a. The cycle length is C.

When slow and fast meet, slow has traveled F + a steps. Fast has traveled 2(F + a) steps. The difference is F + a, which must be a multiple of C (fast completed extra laps around the cycle): F + a = kC for some integer k.

This means F = kC - a. Starting from the meeting point and walking F more steps lands you back at the cycle start (you walk kC - a steps from a point that is a steps past the start, completing k full cycles). Starting from the head and walking F steps also lands at the cycle start. So both pointers arrive at the same node after F steps.

Trace through 1 → 2 → 3 → 4 → 5 → 3 (cycle start at node 3):

From Slide 102, slow and fast meet at node 4.

Phase 2: entry starts at head (node 1), slow stays at meeting point (node 4).

Step	entry	slow	Met?
Start	1	4	No
1	2	5	No
2	3	3	Yes

Both pointers meet at node 3, which is the cycle start. F = 2 steps from head to cycle start, and the algorithm took exactly 2 steps.

When Interviewers Ask About Cycle Detection

The problem typically has two levels. Level 1: detect whether a cycle exists (hasCycle, Slide 102). Level 2: find where the cycle starts (detectCycle, this slide). Some interviewers ask both. Know both phases and be ready to explain the mathematical justification for Phase 2 concisely: “The meeting point and the head are equidistant from the cycle start, modulo the cycle length.”

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

SLIDE 103: Fast-Slow Pointers and Gap Pointers

Finding the Middle: Fast-Slow Technique

Slow moves at speed 1, fast moves at speed 2. When fast reaches the end, slow is at the middle. One pass, no need to know the length.

```
public ListNode middleNode(ListNode head) {
    ListNode slow = head;
    ListNode fast = head;

    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }

    return slow;
}
```

For even-length lists, this returns the second of the two middle nodes (the right-center). If you need the left-center, change the condition to fast.next != null && fast.next.next != null.

Remove Nth Node From End: Gap Pointer Technique

To remove the Nth node from the end without knowing the list length, create a gap of exactly N+1 between two pointers. When the leading pointer reaches null, the trailing pointer is one position before the target.

```
public ListNode removeNthFromEnd(ListNode head, int n) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode fast = dummy;
    ListNode slow = dummy;

    for (int i = 0; i <= n; i++) {
        fast = fast.next;
    }

    while (fast != null) {
        fast = fast.next;
        slow = slow.next;
    }
```

```
    slow.next = slow.next.next;
    return dummy.next;
}
```

The dummy node is essential here. If n equals the list length, the head itself must be removed. Without dummy, removing the head requires a special case. With dummy, slow stops at the dummy node and skips the head through slow.next = slow.next.next.

Intersection of Two Linked Lists

Given two lists that may merge at some node, find the intersection node. The approach: compute the length of each list, advance the longer list's pointer by the length difference, then advance both pointers together. They meet at the intersection.

An elegant alternative: pointer A traverses list A then switches to list B. Pointer B traverses list B then switches to list A. Both pointers travel the same total distance (lenA + lenB), so they arrive at the intersection node simultaneously. If there is no intersection, both reach null at the same time.

Pattern Summary

Problem	Technique	Key Detail
Find middle	Fast-slow (speed 1 and 2)	For left-center, adjust loop condition
Find Nth from end	Gap pointer (N+1 gap)	Use dummy node when head might be removed
Detect cycle	Fast-slow (Slide 102)	If they meet, cycle exists
Find cycle start	Reset one to head, both speed 1 (Slide 102B)	Meet at cycle entry
Find intersection	Equal total traversal	Switch lists after reaching end

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 104: Merging Sorted Linked Lists

Merge Two Sorted Lists (LeetCode 21)

Given two sorted linked lists, merge them into one sorted list by rewiring existing node pointers (no new nodes created).

The approach: use a dummy node and a tail pointer. Compare the heads of both lists, take the smaller one, advance that list's pointer, and advance the tail. When one list is exhausted, attach

the remainder of the other.

```java
public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(0);
    ListNode tail = dummy;

    while (l1 != null && l2 != null) {
        if (l1.val <= l2.val) {
            tail.next = l1;
            l1 = l1.next;
        } else {
            tail.next = l2;
            l2 = l2.next;
        }
        tail = tail.next;
    }

    tail.next = (l1 != null) ? l1 : l2;
    return dummy.next;
}
```

Trace through l1 = 1 → 3 → 5, l2 = 2 → 4 → 6:

Step	l1	l2	Comparison	Attach	Merged so far
1	1	2	1 <= 2	l1 (1)	1
2	3	2	3 > 2	l2 (2)	1 → 2
3	3	4	3 <= 4	l1 (3)	1 → 2 → 3
4	5	4	5 > 4	l2 (4)	1 → 2 → 3 → 4
5	5	6	5 <= 6	l1 (5)	1 → 2 → 3 → 4 → 5
6	null	6	l1 exhausted	remainder l2	1 → 2 → 3 → 4 → 5 → 6

Time O(n + m), space O(1). No new nodes are created. The existing nodes are rewired into the merged order.

Merge K Sorted Lists (LeetCode 23)

When merging K sorted lists, comparing all K heads at each step takes O(K). A min-heap (PriorityQueue in Java) reduces this to O(log K) per extraction.

```java
public ListNode mergeKLists(ListNode[] lists) {
    PriorityQueue<ListNode> heap = new PriorityQueue<>((a, b) -> Intege
r.compare(a.val, b.val));

    for (ListNode head : lists) {
        if (head != null) heap.offer(head);
    }

    ListNode dummy = new ListNode(0);
    ListNode tail = dummy;

    while (!heap.isEmpty()) {
        ListNode smallest = heap.poll();
        tail.next = smallest;
        tail = tail.next;
        if (smallest.next != null) heap.offer(smallest.next);
    }

    return dummy.next;
}
```

Add all K heads to the heap. Extract the minimum, attach it to the result, and add its next node to the heap. Each of the N total nodes is added and removed from the heap once: O(N log K) time, O(K) space.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 105: Worked Example: Reverse Nodes in K-Group

Problem: Reverse Nodes in K-Group (LeetCode 25)

Given a linked list, reverse the nodes in groups of K. If the remaining nodes are fewer than K, leave them as-is.

This problem combines the reversal from Slide 101 with group boundary management. It is rated Hard because the pointer bookkeeping across group boundaries is error-prone under pressure.

Approach

For each group: check if K nodes remain, identify the group boundaries, reverse the group using the three-pointer technique, reconnect the reversed group to the rest of the list, and advance to the next group.

```
public ListNode reverseKGroup(ListNode head, int k) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode groupPrev = dummy;

    while (true) {
        ListNode kth = getKthNode(groupPrev, k);
        if (kth == null) break;

        ListNode groupNext = kth.next;
        ListNode prev = groupNext;
        ListNode curr = groupPrev.next;

        while (curr != groupNext) {
            ListNode next = curr.next;
            curr.next = prev;
            prev = curr;
            curr = next;
        }

        ListNode firstOfGroup = groupPrev.next;
        groupPrev.next = kth;
        groupPrev = firstOfGroup;
    }

    return dummy.next;
}

private ListNode getKthNode(ListNode node, int k) {
    while (node != null && k > 0) {
        node = node.next;
        k--;
    }
    return node;
}
```

The inner while loop is the same reversal from Slide 101, with one difference: instead of reversing until curr is null, it reverses until curr reaches groupNext (the first node of the next group). Setting prev = groupNext before reversing means the last reversed node automatically points to the next group, handling the forward connection.

Trace through 1 → 2 → 3 → 4 → 5 with K = 2:

Phase	Operation	List State
Initial	groupPrev = dummy	dummy → 1 → 2 → 3 → 4 → 5
Group 1: find kth	kth = node 2, groupNext = 3	-
Group 1: reverse [1,2]	prev starts at 3, reverse 1 and 2	dummy → 2 → 1 → 3 → 4 → 5
Group 1: reconnect	groupPrev.next = kth(2), groupPrev = 1	dummy → 2 → 1 → 3 → 4 → 5
Group 2: find kth	kth = node 4, groupNext = 5	-
Group 2: reverse [3,4]	prev starts at 5, reverse 3 and 4	dummy → 2 → 1 → 4 → 3 → 5
Group 2: reconnect	groupPrev.next = kth(4), groupPrev = 3	dummy → 2 → 1 → 4 → 3 → 5
Group 3: find kth	only 1 node remaining, kth = null	break

Result: 2 → 1 → 4 → 3 → 5. Groups [1,2] and [3,4] reversed, node 5 left unchanged (fewer than K remaining).

Why This Problem Matters

It demonstrates that the basic reversal (Slide 101) is a building block, not just a standalone problem. Once you can reverse a segment reliably, you plug it into harder problems. The same reversal subroutine appears in “Reverse Linked List II” (reverse between positions m and n), “Palindrome Linked List” (reverse second half then compare), and “Reorder List” (reverse second half then interleave).

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 106: Wrong vs Right: Linked List Mistakes

Mistake 1: Modifying curr.next before saving it

Wrong: writing curr.next = prev before saving curr.next into a temporary variable. The rest of the list is lost. This is the single most common linked list bug and is extremely difficult to debug because the list appears to work until you try to traverse past the corrupted node.

Better: always save curr.next as the first operation. The reversal order is: save, rewire, advance prev, advance curr. Memorize this sequence.

Mistake 2: Converting to an ArrayList and back

Wrong: copying all node values into an ArrayList, solving the problem on the array, then reconstructing a new linked list. This technically works but interviewers treat it as dodging the actual problem. It uses O(n) space for the array and misses the point: linked list problems test pointer manipulation.

Better: solve in-place using pointer operations. If you need random access (like checking for palindromes), use fast-slow pointers to find the middle, reverse the second half, then compare.

Mistake 3: Forgetting null checks

Wrong: accessing node.next or node.val when node might be null. This produces NullPointerException and is the second most common linked list bug.

Better: guard every pointer dereference. In the while condition, check fast != null && fast.next != null (not just fast.next != null). When traversing, always verify the node exists before accessing its fields.

Mistake 4: Recursive solutions on large lists

Wrong: using recursion for reversal or merging on lists with 100,000+ nodes. Each recursive call adds a stack frame. At depth 100,000, the program crashes with StackOverflowError.

Better: prefer iterative solutions for linked list operations. Iterative reversal is O(1) space. Recursive reversal is O(n) space due to the call stack. The iterative version is also more natural for pointer manipulation.

Mistake 5: Not using a dummy node when the head might change

Wrong: writing special-case logic to handle head removal, head insertion, or head replacement. This adds complexity and is a frequent source of off-by-one errors.

Better: always use a dummy node when the operation might modify the head. It costs one line to set up, eliminates the special case, and you return dummy.next at the end.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 107: How to Present Linked List Solutions in an Interview

Draw the Pointers

Even a rough diagram prevents wiring errors. Before coding, sketch 3-4 nodes with arrows. When explaining reversal, draw the before and after states. When explaining cycle detection, draw the cycle and mark where slow and fast are at each step. Interviewers expect visual reasoning for linked list problems. Coding without drawing first is a red flag.

How to Explain Reversal

“I maintain three pointers: prev, curr, and next. At each step, I save the next node so I don’t lose it, rewire curr to point backward to prev, then advance both prev and curr. After the loop, prev points to the new head.”

This explanation covers the what (three pointers), the why (save next before rewiring), and the result (prev is the new head). It takes 15 seconds and gives the interviewer confidence before you start writing code.

How to Introduce the Dummy Node

“I’ll use a dummy node since the head might change during this operation. This lets me handle every node uniformly without special-casing the head.”

State this proactively. Do not wait for the interviewer to ask about edge cases. Using a dummy node demonstrates experience with linked list problems.

How to Present Floyd’s Algorithm

“I’ll use two pointers at different speeds. If there’s a cycle, the fast pointer will eventually lap the slow pointer and they’ll meet inside the cycle. The time complexity is O(n) and space is O(1).”

If asked why they must meet: “Inside the cycle, fast gains one position per step relative to slow. The gap decreases by 1 each step, so it must reach 0.”

If asked how to find the cycle start: “After they meet, I reset one pointer to the head and advance both at speed 1. They meet at the cycle entry point because the distances from the head and from the meeting point to the entry are congruent modulo the cycle length.”

General Linked List Guidance

Always state whether you are modifying the list in-place or creating new nodes. If the problem does not specify, ask. Some problems require preserving the original list.

State time and space complexity explicitly. Many linked list solutions are O(n) time and O(1) space, and that O(1) space is a strength worth highlighting.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 108: Interview Practice: Linked Lists

Practice Prompt

“Reverse a singly linked list.”

Expected Walkthrough

Step 1, Understand: singly linked, each node has val and next. Reverse all pointers so the list runs in the opposite direction. The old tail becomes the new head. Edge cases: null (return null), single node (return as-is).

Step 2, Approach: iterative three-pointer technique. prev starts at null, curr starts at head. At each step: save next, rewire, advance.

Step 3, Implement: the solution from Slide 101.

Step 4, Verify: trace through a 4-node list. Confirm prev is the new head at the end. State O(n) time, O(1) space.

Follow-Up: "Now reverse only positions 2 through 4 in 1 → 2 → 3 → 4 → 5"

This tests whether you can isolate a segment and apply the reversal subroutine to it. The approach: traverse to node before position 2 (node 1), apply reversal on the sublist from position 2 to 4, reconnect the reversed segment to the prefix (node 1) and suffix (node 5).

Expected result: 1 → 4 → 3 → 2 → 5.

Key insight: the node at position 2 (originally head of the reversed segment) becomes the tail after reversal. You need to save a reference to it before reversing so you can connect it to the suffix.

Common Fresher Mistakes

Mistake	What Goes Wrong	Fix
Rewiring before saving	curr.next overwritten, rest of list lost	Always save curr.next as the first step
Returning head instead of prev	Returns the old head (now the tail), losing most of the reversed list	The new head is prev after the loop ends
Not handling null and single-node	NullPointerException on empty input	Guard: if head == null or head.next == null, return head
Using recursion without noting space	Says O(1) space for recursive reversal	Recursive reversal uses O(n) stack space. State this if using recursion
Losing connection in partial reversal	Reversed segment not connected to prefix/suffix	Save references to both boundary nodes before reversing

Second Practice Prompt: "Detect if a linked list has a cycle"

Expected: Floyd's algorithm from Slide 102. Be prepared for the follow-up: "Where does the cycle start?" (Slide 102B).

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

SLIDE 109: Section Cheat Sheet

Operation Complexity: Linked List vs Array

Operation	Singly Linked	Doubly Linked	Array
Access by index	O(n)	O(n)	O(1)
Insert at head	O(1)	O(1)	O(n)
Insert at position (given node)	O(1)	O(1)	O(n)
Delete at position (given node)	O(n) to find prev	O(1)	O(n)
Find length	O(n)	O(n)	O(1)
Reverse	O(n)	O(n)	O(n)

Template Quick Reference

Technique	Template Summary	Time	Space
Dummy node	dummy = new ListNode(0); dummy.next = head; ... return dummy.next	-	O(1)
Iterative reversal	prev/curr/next: save, rewire, advance prev, advance curr	O(n)	O(1)
Floyd's cycle detection	slow speed 1, fast speed 2, meet = cycle exists	O(n)	O(1)
Find cycle start	After meeting, reset one to head, both speed 1, meet at entry	O(n)	O(1)
Find middle	slow speed 1, fast speed 2, fast reaches end = slow at middle	O(n)	O(1)
Nth from end	Gap of N+1 between fast and slow, with dummy	O(n)	O(1)
Merge two sorted	Dummy + tail, compare heads, attach smaller, attach remainder	O(n+m)	O(1)
Merge K sorted	PriorityQueue of K heads, extract min, add its next	O(N log K)	O(K)

Pattern Recognition Signals

Problem Signal	Technique	Key Detail
"reverse" (any form)	Three pointers: prev, curr, next	Save before rewire. Practice until automatic
"cycle", "loop", "circular"	Floyd's tortoise and hare	Two speeds for detection, equal speed for start
"middle", "split in half"	Fast-slow pointers	Fast at 2x speed reaches end when slow at middle

Problem Signal	Technique	Key Detail
“Nth from end”, “remove from end”	Gap pointer with dummy	Gap of N+1, dummy handles head removal
“merge sorted lists”	Dummy + comparison loop	Attach remainder when one list exhausted
“reverse in groups”	Reversal subroutine + group boundary management	getKthNode checks if enough nodes remain
“palindrome linked list”	Find middle + reverse second half + compare	Combines two techniques

Common Interview Mistakes

Mistake	Wrong	Better
Lose rest of list	Rewire before saving curr.next	Save next first, always
Convert to array	Copy to ArrayList, solve, rebuild	Solve in-place with pointers
Stack overflow	Recursive solution on 100K nodes	Iterative for O(1) space
Head edge cases	Special-case head removal/insertion	Use dummy node
Claim O(1) space for recursion	“Iterative and recursive are equivalent”	Recursion uses O(n) call stack

How This Section Connects to Other Sections

Section	Connection
Two Pointers (Section 6)	Fast-slow on linked lists is the same-direction two-pointer pattern applied to a non-indexed structure
Sorting (Section 9)	Merge sort on linked lists uses find-middle (split) and merge-two-sorted (combine). Linked lists are ideal for merge sort because the merge step requires no extra space
Stacks (Section 10)	Palindrome check alternative: push first half onto stack, compare with second half
Recursion (Section 12)	Reversal and merging have recursive variants. Know both, prefer iterative for space

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina